

Recent Trends in Mixed-Mode Programming for High-Performance Computing

Group 2 [13-24] · Innovative Examination · 2026-27 · Microprocessor

Group 2 [13-24] – Raunak SIngh (13) · Rishit Singh (14) · Rudrapratap Singh (15) · Sakshi Singh (16) · Udaypratap Singh (17) · Vicky Singh (18) · Vishesh Singh (19) · Divyajoyt Sinha (20) · Deepesh Sonar (21) · Vedant Sud (23) · Dhvani Tandon (24)

Guide: Ms. Tanmayi Nagale · Assistant Professor

CONTEXT: THE MIXED-MODE TURN

MPI+X is the de facto standard for distributed parallel programming. The open question is which X – OpenMP, tasks, GPU offload, or PGAS – and how to combine them without drowning in complexity.

OPENING MOVEMENTS: THE FULL ARC

Eight movements from MPI+X to PGAS and GPU persistence – every section feeds the same argument: mixed-mode is how HPC scales.

01 Introduction & Context

Why mixed-mode became HPC's default: MPI+X, node-level OpenMP, and the scaling wall.

02 Why Hybrid MPI+OpenMP

Flat MPI exhausts the network interface; OpenMP threads shrink PME message traffic.

03 Task-Based Runtimes

Dependency-driven DAG scheduling replaces static decomposition; comms overlap with compute.

04 GPU Offload & Accelerators

CUDA/SYCL/HIP offload; GPU-resident execution becomes the default in 2025.

05 One-Sided & PGAS

DiOMP folds PGAS into OpenMP via GASNet-EX – measured wins follow.

MIXED-MODE: DE FACTO HPC PARADIGM

MPI remains the distributed-memory backbone; OpenMP fills the intra-node gap as core counts climb.

- MPI+X is the de facto standard for distributed-memory parallel programming, per the DiOMP study (arXiv:2409.02830).
- Purely MPI schemes share one network interface per node; OpenMP threads inside a rank cut inter-node message counts.
- Multi-level MPI+OpenMP is needed to keep scaling near the strong-scaling limit, around ~200 particles/core.
- OpenMP scales well to 12-24 threads on Intel and 6-8 on AMD, matching typical per-socket core counts.
- Hybrid runs add overhead at small process counts, but the payback is pronounced in highly parallel runs.
- NUMA effects on 2-4 socket nodes make intra-node threading the default way to avoid memory-bus bottlenecks.

WHY HYBRID: FLAT MODELS HIT THE WALL

The multi-core trend has pushed nodes to 2-4 CPUs and up to 64 cores. Flat MPI exhausts the network interface first; pure OpenMP cannot cross the socket. Hybrid MPI+OpenMP is the only path that scales.

FLAT MPI VS PURE OPENMP: TWO CEILINGS

Hybrid splits the difference: OpenMP shrinks the number of MPI messages; MPI gives OpenMP a way across the node boundary.

Flat MPI

Every core a rank floods one network interface;
PME's all-to-all 3D FFT makes inter-node hops
the bottleneck.



vs

Pure OpenMP

Threads inside one node dodge the network, but
NUMA caps scaling at 12-24 Intel / 6-8 AMD. No
cross-node reach.

HYBRID AS LAYERING: MPI ACROSS NODES, OPENMP ACROSS CORES

Flat MPI exhausts the network interface; pure OpenMP cannot cross sockets. Hybrid splits the job: MPI for inter-node coordination, OpenMP for intra-node threading.

- Each MPI rank owns one node's memory and talks across the network; OpenMP threads inside the rank share that memory and never send messages.
- Fewer ranks per node means fewer messages on the single network interface – PME's 3D FFT all-to-all is otherwise the scaling bottleneck.
- OpenMP scales to 12-24 threads on Intel and 6-8 on AMD, matching typical 2-4 socket node topologies and cutting NUMA crossings.
- Hybrid runs add overhead at small process counts, but the payback is pronounced at scale – around ~200 particles per core.
- GROMACS mdrun follows this layering: -ntmpi processes for nodes, -ntomp threads per rank, dedicated PME ranks above 12-16 ranks.

TASK-BASED RUNTIMES: DEPENDENCIES OVER DECOMPOSITION

Move from static MPI ranks to dynamic DAG scheduling: runtimes express data dependencies, overlap communication and computation, and load-balance without hand-written message passing.

TASK-BASED: DEPENDENCIES, NOT BARRIERS

Programmer names dependencies; runtime orders execution across nodes and devices.

Define work as tasks

1

Each unit of work declares its inputs and outputs; runtime infers ordering from the dependency graph.

Build the graph

2

At each step the runtime assembles a DAG; ready tasks launch asynchronously as dependencies complete.

Heterogeneous placement

3

Work lands on CPU or GPU by cost; GROMACS auto-balances PP and PME across ranks and devices.

Overlap and communicate

4

Direct GPU-GPU transfers replace host staging; GROMACS 2025 enables this by default on supported setups.

Cut launch overhead

5

CUDA graphs (2023) schedule entire graphs at once, cutting per-step launch costs in fast-iterating runs.

GPU OFFLOAD: FROM STAGED COPIES TO RESIDENT KERNELS

GROMACS 2025 enables direct GPU communication by default; GPU-resident mode keeps force and coordinate data on the device, making the CPU a partner rather than the pipeline.

GPU OFFLOAD STACK: DIRECTIVES TO SILICON

Persistent kernels and resident data shrink launch and copy overhead – the stack now spans pragmas to vendor runtimes.

Application Directives

Portable offload via pragma-based models

OpenACC

OpenMP target

pragma offload

Runtime Models

Device control through vendor and open standards

CUDA

SYCL

HIP

OpenCL

Kernel Scheduling

Persistent kernels and CUDA graphs cut costs

persistent kernels

CUDA graphs

async launch

Data Movement

GPU-resident mode avoids per-step CPU transfers

GPU-resident

force-offload

direct GPU comm

Hardware Abstraction

Vendor accelerators behind a common stack

NVIDIA

AMD

Intel

The key shift: persistent kernels and GPU-resident mode treat the accelerator as a peer, not a peripheral, cutting per-step transfers and launch latency.

ONE-SIDED & PGAS: REMOTE MEMORY WITHOUT THE HANDSHAKE

DiOMP fuses PGAS into OpenMP via LLVM + GASNet-EX: bandwidth up 25%, latency down 45% vs MPI+OpenMP.

ONE-SIDED PGAS: REMOTE PUT/GET WITHOUT HANDSHAKE

DiOMP lowers OpenMP onto GASNet-EX PGAS: 25% higher bandwidth, 45% lower latency than MPI+OpenMP on Ookami and Perlmutter.

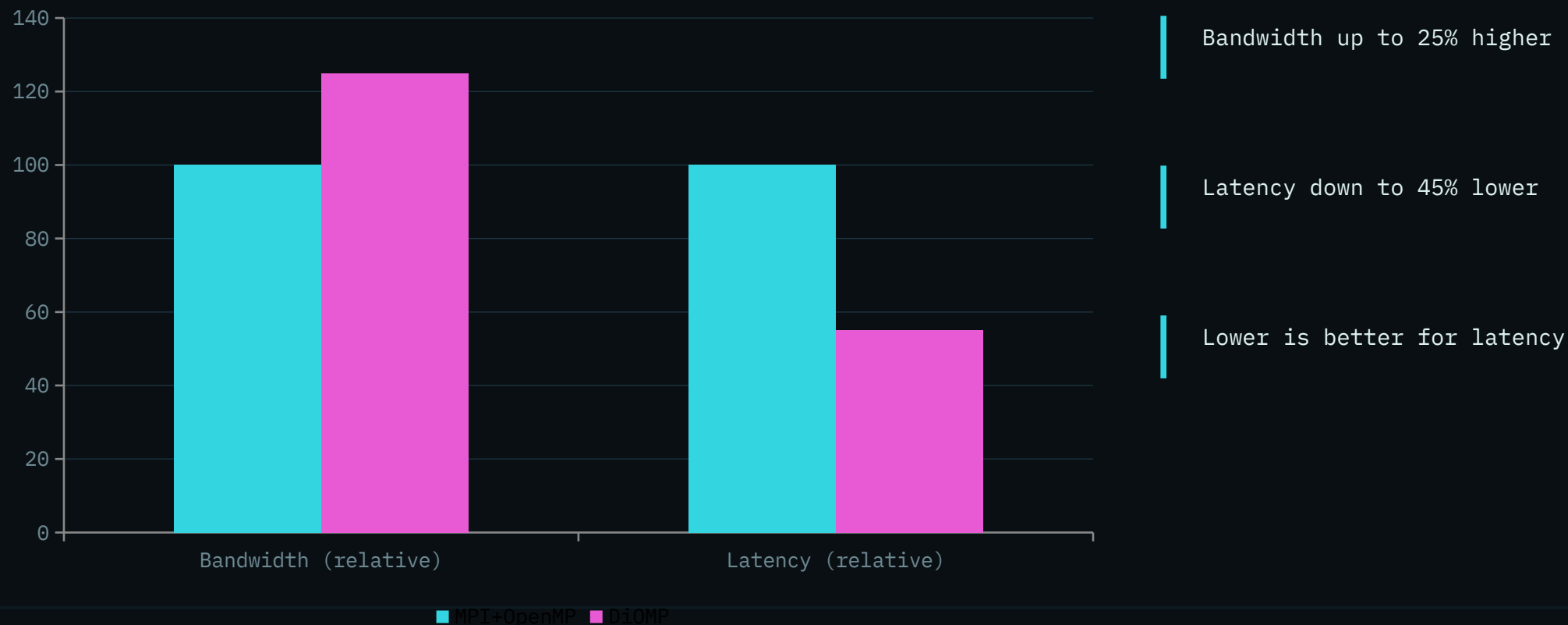


PERFORMANCE EVIDENCE: WHO WINS AT SCALE

GROMACS tuning reports and DiOMP's 25% bandwidth / 45% latency results turn the hybrid-vs-PGAS debate into a measurement.

DIOMP: 25% HIGHER BANDWIDTH, 45% LOWER LATENCY

PGAS-style OpenMP via LLVM and GASNet-EX beats MPI+OpenMP on two key axes at scale.



WHY HYBRID SCALES: MEASURED GROMACS LIMITS

GROMACS tuning documentation turns the scaling story into numbers: thread ceilings, PME bottlenecks, and the hybrid crossover.

✓

Strong-scaling rule: keep ~200 particles/core; past that, flat-MPI efficiency drops and hybrid ranks win.

✓

OpenMP scaling ceiling: 12-24 Intel / 6-8 AMD threads; beyond it, add MPI ranks, not threads.

✓

Above 16 ranks, PME's all-to-all 3D-FFT bottlenecks; dedicated PME ranks are the hybrid split that restores scaling.

✓

Hybrid adds overhead at small process counts; the message-count payoff only appears at high core counts – the crossover is the argument.

✓

On multi-GPU nodes, GPU-resident mode plus direct GPU comm removes CPU staging; rank-per-GPU mapping sets the ceiling.

DIRECTIONS: WHERE MIXED-MODE GOES NEXT

PGAS concepts folding into OpenMP, GPU-resident execution becoming default – the productive path forward for distributed-memory HPC.

DIRECTIONS: FUSION → PERSISTENCE → EXASCALE

DiOMP proved PGAS-in-OpenMP at 25% higher bandwidth; GPU-resident and direct GPU comm are now defaults; the pipeline runs to exascale PME decomposition.

	2024	2023-25	2026+
Runtime Fusion	LLVM + GASNet-EX PGAS primitives inside OpenMP, no MPI boilerplate	Ookami + Perlmutter Microbenchmarks and kernels validated on both	25% bandwidth win 45% lower latency vs MPI+OpenMP
GPU Persistence	GPU-resident mode Update and constraints stay on GPU; CPU idle cut	Direct GPU comm Default since 2025; bypasses CPU staging	CUDA graphs Added 2023; trims launch overhead
Exascale Systems	PME decomposition Distributed 3D-FFT via cuFFTMp/HeFFTe	HIP PME path AMD CDNA support since 2026 release	NVLINK nodes GPU-aware MPI for multi-node resident runs

REFERENCES & CLOSE: THE EVIDENCE TRAIL

Every performance claim in this deck traces to a cited source – DiOMP, GROMACS tuning guides, and PGAS results – before the final synthesis.

SOURCE RECORD: MIXED-MODE EVIDENCE

Primary documentation and peer-reviewed results backing every claim in this deck.

Baodi Shan et al. published “Towards a Scalable and Efficient PGAS-based Distributed OpenMP” as arXiv:2409.02830 [cs.DC] on 4 Sep 2024.

DiOMP, built on LLVM and GASNet-EX, achieved 25% higher bandwidth and 45% lower latency than MPI+OpenMP on Ookami and Perlmutter.

GROMACS Project documents “Getting good performance from mdrun” in the current release.

This authoritative tuning guide covers thread-MPI versus MPI, OpenMP threads, GPU offload flags, and PME rank counts.

GROMACS Project. “Getting good performance from mdrun.” GROMACS 5.1.1 documentation.

This document supplies the baseline for SIMD kernel selection, -ntmpi/-ntomp/-npme controls, and strong-scaling guidance around 200 particles/core.

GROMACS Reference Manual. Domain decomposition and parallelization chapters.

Defines PP/PME split, dynamic load balancing, update groups, and GPU-resident constraints.

MIXED-MODE: FUSE, OFFLOAD, REMOTE — THEN SCALE

DiOMP shows PGAS one-sided beats MPI+OpenMP: up to 25% higher bandwidth, 45% lower latency. GROMACS scales with MPI+OpenMP plus GPU-resident offload and direct GPU communication. Path forward: hybrid MPI+OpenMP, task-based dependencies, GPU offload, and one-sided PGAS on hot paths.

MEASURE ON YOUR NODE — START
HYBRID TODAY